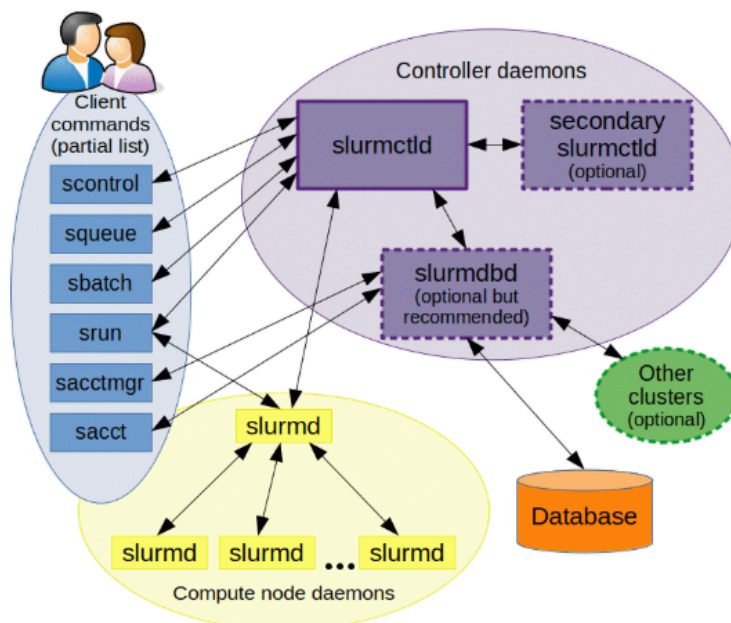


Viswam Compute

Ada Cluster Summary:

- **Nodes:** 92 Boston SYS-7048GR-TR
- **Processors:** Dual Intel Xeon E5-2640 v4 (40 virtual cores/node)
- **RAM:** 128 GB DDR4 ECC/node
- **GPUs (per node):**
 - 4x Nvidia GeForce GTX 1080 Ti (14336 CUDA cores, 44 GB GDDR5X)
 - OR 4x Nvidia GeForce RTX 2080 Ti (17408 cores, 44 GB GDDR6)
- **Connectivity:** Gigabit Ethernet
- **Storage (per node):** 1.8 TB local scratch + 960 GB local SSD scratch
- **OS:** Ubuntu 18.04 LTS
- **Job Management:** SLURM
- **Theoretical Peak Performance:** 70.66 TFLOPS (CPU) + 4588 TFLOPS (FP32 GPU)

Slurm is an open source, fault-tolerant, and highly scalable cluster management and job scheduling system for large and small Linux clusters.



Partitions , Accounts and QoS(Quality of Service)

Partitions can be considered as collection of nodes:

- Partitions act as job queues, grouping nodes.
- Each partition has constraints: job size, duration, authorized users.
- Jobs are prioritized and assigned nodes until all resources are utilized.
- Once nodes are allocated, users can start parallel tasks through job steps.
- Job steps are configurable within allocated resources (e.g., single step uses all nodes, or multiple independent steps use portions).

We viswam have access to edcs partition which is ,

Nodes: gnode[124-127]

Gres: gpu:4

A SLURM account is like a bank account, and all users belong to at least one account. The allocated resources to a job are charged to the job's specified account.

We viswam have access to research account with medium allowed QoS.

QoS	MaxCPUsPerUser	MaxTRESPerUser	MaxJobsPerUser	MaxSubmitJobsPerUser	MaxWall	MaxTresPerJob
low	10	gres/gpu=1	1	4	4-00:00:00	gres/gpu=1
medium	40	gres/gpu=4	4	8	4-00:00:00	gres/gpu=4
normal	Account limits	Account limits	Account limits	Account limits	Infinite	gres/gpu=4

So as per medium QoS we have below **User Resource Limits:**

- **Maximum GPU Usage:** 4 GPUs
- **Maximum Concurrent Jobs:** 4 jobs
- **Maximum Submitted Jobs:** 8 jobs
- **Maximum Wall Time per Job:** 4 days
- **Maximum GPUs per Job:** 4 GPUs

sacctmgr show assoc account=viswamai

Above command used to see all users associated with your account and the resources allocated to it.

Storage

The cluster provides four types of file storage to users. They are referred here as /home, /share6, /scratch and /ssd_scratch.

The /home located at /home/\$USER is an NFS storage and has a disk quota of 25 GB. This space can be used to store your source code and to build your executables. The data store on /home is backed up every day. The /share6 is RAID6 storage available on the master node and has a quota of 100 GB. The /scratch is for storing temporary files created during job run time. The /ssd_scratch is for storing temporary files that required high-speed disk I/O access.

Space	Purpose	Visibility	Backup	Quota
/home	Software installation space, storing codes and small files	Master and compute nodes	Yes	25 GB
/share6	Long-term storage	Master node only	No	100 GB
/scratch	Temporary storage for large files	Local disk attached to each compute node	No	None
/ssd_scratch	Temporary storage for jobs that require fast I/O	Local disk attached to each compute node	No	None

Basic Flow while working with HPC:

Login through:

```
$ ssh -X user_name@ada.iiit.ac.in
```

This takes you to home of your account , to transfer files between your local device (laptop/pc) into HPC u can use

```
$ scp -r local_directory user_name@ada.iiit.ac.in:
```

```
$ rsync -avz local_directory user_name@ada.iiit.ac.in:
```

And for to Ada to local machine:

```
$ scp -r user_name@ada.iiit.ac.in:remote_directory .
```

```
$ rsync -avz user_name@ada.iiit.ac.in:remote_directory .
```

****Both `scp` and `rsync` transfers files locally or over network. If the transfer is interrupted, `rsync` has the ability to continue from where it left off when invoked again.**

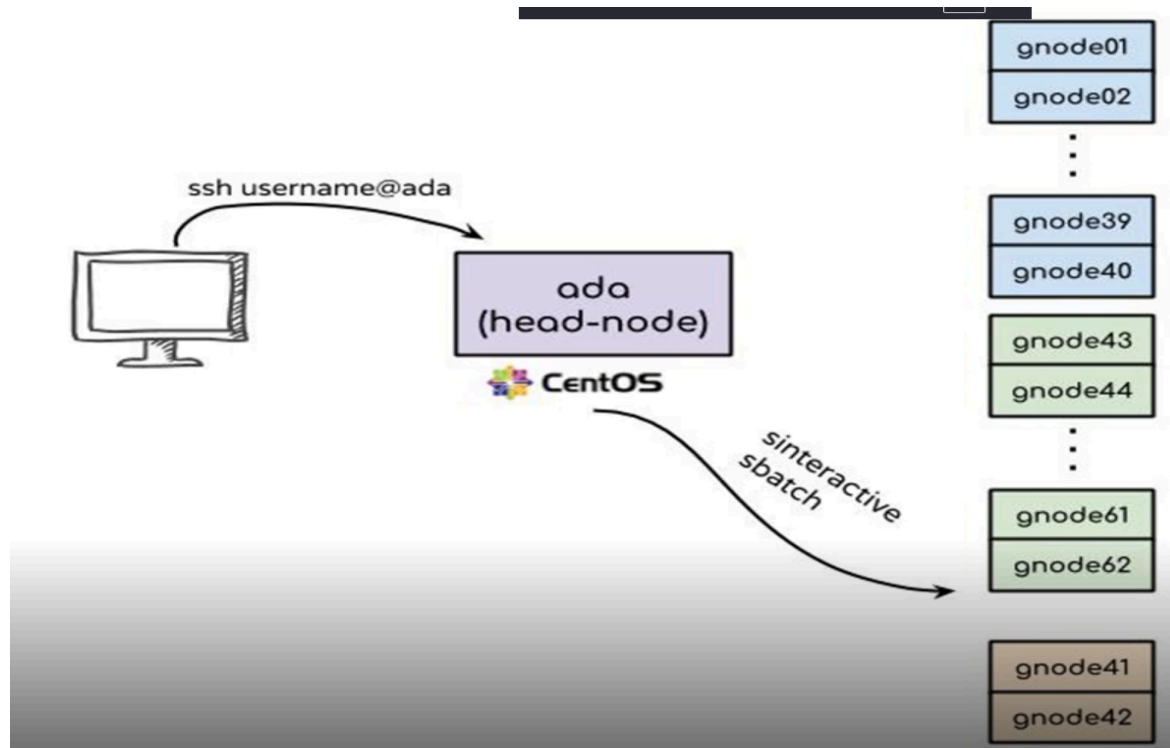
Basic Commands used often:

Command	Description	Example
sbatch	Submit a batch job	sbatch job_script.sh
sinteractive	Submit an interactive job	sinteractive -c 2 -g 1
scancel	Cancel a job	scancel job_id
squeue	List all current jobs for an user	squeue -u username
sacct	Statistics of a completed job	sacct -j job_id --format=user,jobid,jobname, partition,state,time,start,end,e lapsed,allocres,ncpus,nodeli st
sinfo	Display information about nodes and partitions	sinfo -a

Arguments which can be used for much efficient allocation:

- `-n`: Number of nodes
- `-g`: Number of GPUs
- `-c`: Number of CPUs
- `-m`: Memory size
- `-exclusive`: Allocates the node exclusively to you, preventing others from using it.
- `-w`: Assigns a specific node

The Below figure shows the general flow to access the compute nodes.



Case Study:

Compute needed to finetune a 1B parameter model with 100 million tokens with the available compute of 4 nvidia L40s. With Detailed TFLOPs calculations.

-This fine-tuning estimate relies on BF16 precision, an assumed GPU efficiency of 35-50% of peak TFLOPs, and a single training epoch.

Calculation:

1. **GPU Power:** Four NVIDIA L40S GPUs provide a theoretical peak of 2,920 BF16 TFLOPs (4 x 730 TFLOPs/GPU).
2. **Workload:** Fine-tuning a 1 billion parameter model on 100 million tokens requires approximately 600,000 TFLOPs (6 x parameters x tokens).

Why x6??

This is for 2 x parameter x tokens for forward pass and 4 x parameter x tokens for backward pass (double of forward due to gradient descent)

3. Estimated Time:

- **Optimistic (50% efficiency):** 600,000 TFLOPs / 1,460 TFLOPs ($2920 * 0.5$) $\approx$ 411 seconds (6.8 minutes).
- **Conservative (35% efficiency):** 600,000 TFLOPs / 1,022 TFLOPs ($2920 * 0.35$) $\approx$ 587 seconds (9.8 minutes).

This results in a 7 to 12 minute training window per epoch.

Other factors affecting time: Data loading speed, optimized software stack, and inter-GPU communication.